Stream Mining on E-Commerce Event Data

RetailRocket: Next-Category Prediction and Purchase Intent Detection
Using Adaptive Streaming Models with Concept Drift Analysis

Introduction

This project applies streaming machine learning to the RetailRocket e-commerce dataset, evaluating two prediction tasks in an online, prequential setting:

- **Task A — Next parent category prediction.** Given a user's browsing history and global stream state, predict which product category the user will interact with next. This is a multi-class problem with 283 classes.
- **Task B — Purchase intent prediction.** Given a user's session so far, predict whether the current event is a transaction (purchase). This is a binary classification problem with heavy class imbalance: transactions constitute approximately 0.81% of all events.

All models are evaluated under the prequential (test-then-train) protocol using the River library for online learning. Each arriving event is first used for prediction and metric update, then used to train the model. No batch re-training or look-ahead is permitted. In addition, we simulate five types of concept drift and evaluate how each model responds.

A key methodological decision is that both the item identifier and the product category are hidden from the model at prediction time. The model must predict the category purely from user-level and global-level features, without knowing which specific item the user is about to click. This reflects the realistic deployment scenario where a recommender system must decide what to show a user *before* the user makes a choice.

Data Preparation

Source Data

The RetailRocket dataset consists of three files:

- `events.csv` — approximately 2.76 million user events, each with a timestamp (Unix milliseconds), visitor identifier, event type (`view`, `addtocart`, or `transaction`), and item identifier.
- `item_properties_part1.csv` and `item_properties_part2.csv` — time-stamped item metadata. Properties include `categoryid` and property `790` (interpreted as price). Values change over time as items are recategorized or repriced.
- `category_tree.txt` — a static hierarchy mapping each `categoryid` to a `parentid`.

Point-in-Time Merging

To avoid future information leakage, item properties are joined to events using `pandas.merge_asof` with `direction='backward'`: for each event, the most recent known value of `categoryid` and `property_790` for that item, recorded at or before the event timestamp, is used.

Initial inspection revealed that approximately 30% of rows remained without a category or price after the backward merge. This occurred because the item-properties files sometimes record an item's first property *after* the item's first appearance in the events file. To recover these values, a second `merge_asof` with `direction='forward'` was applied to rows still missing values, and the forward result was used as a fallback. This is justified because we are filling in a static product attribute (its category or price), not leaking future user behaviour.

Table Construction

Two tables were produced:

- **Table 1** retains the raw `categoryid` (1,139 distinct values).
- **Table 2** joins the category tree, replacing `categoryid` with `parent_category` (283 distinct values). Where no parent exists, the original `categoryid` is used.

All subsequent work uses Table 2 exclusively. The reduction from 1,139 to 283 classes was motivated by two observations: (1) tree-based models are structurally bottlenecked on high-cardinality targets because each leaf can output only one class, and (2) evaluation metrics such as top-1 accuracy become noisy and uninformative when the number of classes is very large relative to the per-class training data.

Feature Engineering

Design Principle: Item Hidden at Prediction Time

At prediction time, both `parent_category` (the target) and `itemid` are hidden from the model. The price (`property_790`) is also dropped because it is a per-item attribute: 97.3% of items have a near-deterministic mapping from price to category, so exposing the price would effectively reveal the target.

The raw identifiers `visitorid` and `itemid` are used only to update stateful components *after* the prediction is made. They are never passed as features to any model.

Engineered Features

All features fall into three categories: per-user features derived from the user's event history, transition features derived from category-to-category patterns, and global/temporal features.

3.2.1 Per-User Session Features

A `UserSessionTracker` maintains cumulative event counts and timing per visitor:

- `prior_view_count`, `prior_cart_count`, `prior_transaction_count`: cumulative counts of each event type for this user.
- `cart_to_view_ratio`: the ratio of cart events to view events, computed as `prior_cart_count`/ max(1, `pri`
  This is a strong indicator of purchase intent.
- `log_time_since_last`: $\log(1+\Delta t)$ where Δt is the time in seconds since the user's previous event. The logarithmic transform prevents returning users (with gaps of hours or days) from dominating tree splits.
- `is_new_session`: binary flag, set to 1 if `log_time_since_last` exceeds 30 minutes (1,800 seconds). Session boundaries mark a qualitative change in user behaviour.
- `events_in_session`: a counter that resets on each session boundary.

3.2.2 Pattern Mining Features

A `PatternMiner` maintains a per-user sliding window of the last 5 parent categories visited, and a global sliding window of the last 2,000 events:

- `last_seen_parent`: the parent category of the user's most recent prior event. This is the single strongest predictor for the next-category task, due to session autocorrelation.
- `recent_diversity`: the number of distinct categories in the user's last 5 events.
- `trending_last_seen`: the fraction of the global window (last 2,000 events) that belongs to the user's last-seen category. Captures whether the user's recent interest is globally popular.

3.2.3 Transition Features

A `TransitionMatrix` maintains a streaming first-order Markov model: for each pair (last_category, next_catego it counts how often this transition has been observed globally. From this it derives:

- `transition_top_prob`: the probability of the most likely next category given the user's last category. Low values indicate a fork in behaviour.

3.2.4 Graph Features

A `WeightedBipartiteGraph` maintains a fading window of 5,000 user–item interaction edges, weighted by event type (view = 1, add-to-cart = 3, transaction = 5). Only user-side features are exposed at prediction time:

- `user_degree`: the number of distinct items this user has touched in the recent window.
- `user_total_weight`: the sum of edge weights for this user.

Item-side features (`item_degree`, `user_item_repeat`, `user_user_similarity_max`) were removed because computing them requires knowing the current item, which is hidden at prediction time.

3.2.5 Temporal and Anomaly Features

- `hour_of_day`: extracted from the event timestamp as (`timestamp_ms` // 3,600,000) mod 24. E-commerce traffic exhibits strong diurnal patterns.
- `anomaly_score`: the streaming anomaly score from a Half-Space Trees (HST) detector with 25 trees, height 15, and a window size of 250. The HST is trained on the non-leaky subset of features (session features and user-graph features). Its score is passed to the ARF Big model as an additional input feature.

Lean Feature Views

Two task-specific subsets of 5 features each were defined for shallow tree models:

- **Lean for next-parent**: `last_seen_parent`, `transition_top_prob`, `prior_view_count`, `is_new_session`, `log_time_since_last`.
- **Lean for intent**: `prior_view_count`, `prior_cart_count`, `prior_transaction_count`, `cart_to_view_ratio`, `is_new_session`.

The rationale is that shallow trees perform better with fewer high-signal features than with many noisy ones. Empirically, ARF Lean (5 trees, depth 8, lean features) outperformed ARF Big (15 trees, full features) on the next-parent task, confirming this hypothesis.

Missing Value Handling

Missing `property_790` values (which occur when no price record exists for an item at the time of the event) are imputed with the streaming mean of all prices observed so far, using River's `stats.Mean()`. Rows where `parent_category` is missing are skipped for the next-parent task (we cannot score a prediction when the ground truth is unknown) but still processed for the intent task.

Models

Task A: Next Parent Category (6 Contestants)

4.1.1 Baselines

Last seen + trending fallback. Predicts the parent category of the user's most recent prior event. On the user's first event (cold start), it falls back to the most popular category in the last 2,000 global events (the "trending" fallback). The trending distribution is maintained as a sliding window with eviction, so it adapts to popularity shifts.

Transition matrix + trending fallback. Uses the streaming first-order Markov model to predict the most common next category given the user's last category. Falls back to trending on cold start.

4.1.2 ML Models

ARF Big (Adaptive Random Forest, 15 trees, default depth, full feature set). This is River's `forest.ARFClassifier` with `n_models=15`. Each tree in the ensemble has its own internal ADWIN drift detector; when ADWIN flags a persistent increase in a tree's prediction errors, the tree is replaced by a background learner that has been training in parallel.

ARF Lean (Adaptive Random Forest, 5 trees, max depth 8, lean features). Uses only the 5 lean features defined above. Tests the "less is more" hypothesis: fewer high-signal features with shallower trees.

HAT (Hoeffding Adaptive Tree, single tree, lean features). River's `tree.HoeffdingAdaptive-TreeClassifier` with `grace_period=200` and `delta=1e-5`. A single tree with built-in ADWIN drift detection at every internal node. When ADWIN fires at a node, only the affected subtree is replaced, allowing local drift adaptation without rebuilding the entire tree.

4.1.3 Hybrid Model

Hybrid (TM + ARF Lean). A cascading predictor: if the transition matrix's top probability exceeds a threshold of 0.1, its prediction is used; otherwise, ARF Lean's prediction is used; if ARF Lean has not yet grown any splits, the trending fallback is used. This design is intended to use the strong baseline where it is confident and ML where the baseline is uncertain.

4.1.4 Cold-Start Policy

Every model scores on every event, including the 51.3% of events that are first-time users. The trending fallback (`GlobalTrendingColdStart`) predicts the most popular category in the last 2,000 events. For the baselines (Last seen, Transition matrix), the fallback fires whenever the user has no prior history. For the ML models, it fires only in the very first events of the stream before any tree has grown a split ($\sim$200 events). This ensures a fair comparison: no model can inflate its accuracy by abstaining on hard cases.

Task B: Purchase Intent (5 Contestants)

4.2.1 Baselines

Always-zero. Always predicts no transaction. Reports 0.5 for ROC-AUC scoring and 0 for F1 scoring.

Cart heuristic. Predicts 1 (transaction) if the user has any prior add-to-cart event in their history, else 0.

4.2.2 ML Models

All three ML models are wrapped in River's `imblearn.RandomOverSampler` with `desired_dist={0: 0.7, 1: 0.3}`. This is necessary because, at the natural ∼99/1 class ratio, Hoeffding-based trees never find a split worth making: every leaf stays nearly pure on class 0, and `predict_proba_one` returns the same value for every event, producing ROC-AUC of exactly 0.5 and F1 of 0.0. The oversampler resamples the training stream so the trees see roughly a 70/30 mix. It affects only training; prediction and evaluation use the original stream.

ARF Big, **ARF Lean**, and **HAT** use the same architectures as in Task A, with the lean intent feature set for ARF Lean and HAT.

Evaluation Metrics

For Task A (multi-class, 283 classes), three metrics are reported:

- **Top-1 accuracy**: the fraction of events where the model's single best guess matches the true category.
- **Top-5 accuracy (Hit Rate @ 5)**: the fraction of events where the true category appears among the model's top 5 ranked predictions. This reflects how a real recommender deploys, showing users a ranked list rather than a single guess.
- **MRR (Mean Reciprocal Rank)**: if the true category appears at rank r in the model's ranked list, the reciprocal rank is $1/r$. The MRR is the mean of these reciprocal ranks across all events. It penalizes lower ranks more than top-K accuracy.

For Task B (binary, imbalanced):

- **ROC-AUC**: the probability that a randomly chosen transaction event receives a higher predicted score than a randomly chosen non-transaction event. Threshold-independent.
- **F1 score**: the harmonic mean of precision and recall at the default 0.5 decision threshold.

Plain accuracy is not reported for Task B because a trivial "always predict 0" model achieves 99.2% accuracy.

Results

The evaluation processes 500,000 events from `table2_stream.csv` in prequential order.

Task A: Next Parent Category

The two baselines and the Hybrid converge to nearly identical lines. The ML models are substantially worse. This result is explained by two properties of the data:

Session autocorrelation. Among warm events (where the user has prior history), 80% of consecutive event pairs share the same parent category. The Last seen baseline directly exploits this by copying the previous category. Tree-based models would need to learn the rule "predict

Model	Top-1	Top-5	MRR
Last seen + trending fallback	0.44	0.52	0.49
Transition matrix + fallback	0.44	0.55	0.49
Hybrid (TM + ARF Lean)	0.44	0.55	0.50
ARF Lean (5 trees, lean)	0.18	0.36	0.28
ARF Big (15 trees, full)	0.14	0.31	0.23
HAT (single tree, lean)	0.09	0.25	0.18

Table 1: Task A results (500,000 events, item hidden at prediction time).

the value of `last_seen_parent`" for each of 283 distinct values, which axis-aligned splits cannot express compactly.

Cold-start dominance. 51.3% of all events are first-time users. For these events, no user-specific features are available; all models effectively fall back to the global trending distribution, which achieves only ∼4% top-1 accuracy. This pulls all models down from their warm-event accuracy.

The Hybrid does not provide meaningful lift over the baselines because: (1) when the user is warm, the transition matrix has confidence ≥ 0.1 almost always, so it is used; (2) when the user is cold-start, ARF Lean has no informative features and essentially predicts the global prior.

ARF Lean outperforms ARF Big, confirming the "less is more" hypothesis: with 283 classes and limited per-class data, fewer high-signal features produce better splits than many noisy features.

Task B: Purchase Intent

Model	ROC-AUC	F1
Cart heuristic	0.93	0.16
ARF Big (oversampled)	0.90	0.19
ARF Lean (oversampled)	0.88	0.18
HAT (oversampled)	0.81	0.10
Always-zero	0.50	0.00

Table 2: Task B results (500,000 events, item hidden at prediction time).

The cart heuristic achieves the highest ROC-AUC (0.93) because it cleanly partitions users into "with prior cart" and "without prior cart." The actual transaction rate differs sharply between these two groups, producing near-optimal ranking from a binary signal.

The ML models beat the cart heuristic on F1 (ARF Big: 0.19 vs. cart: 0.16). The ML models are more selective about *when* a cart-positive user will actually transact, leveraging features such as `cart_to_view_ratio`, `prior_view_count`, and session timing. HAT underperforms both ARFs, confirming that ensembles help for this binary task.

Drift Experiments

Setup

The first 200,000 events from `table2_stream.csv` are loaded into memory. Drift is injected at row 100,000 by mutating the post-drift portion of the event list. Each scenario produces a fresh

copy of the data and a fresh set of models. Accuracy (or ROC-AUC / F1) is recorded every 5,000 events and plotted with a vertical marker at the drift point.

Next-Parent Drift Scenarios

6.2.1 Repeat-Row Drift (Mild)

At row 100,000, the next 5,000 events are replaced with copies of the event at the drift point. This simulates a burst of identical traffic (e.g., a bot or a viral product page).

Observation: All models jump upward during the duplicate run. The duplicated event repeats the same user and category, so Last seen is correct on nearly 100% of these events. After the duplicates end at row 105,000, accuracy gradually returns to baseline. All models exploit short-term repetition equally.

6.2.2 Category-Shuffle Drift (Medium)

At row 100,000, a random permutation is applied to all subsequent `parent_category` values. Specifically: the set of all distinct categories is collected, a random shuffle is generated (with seed 42), and a bijective mapping from old to new labels is applied to every post-drift event.

Observation: No visible effect on any model. The permutation is *consistent* — if category 12 becomes 87 everywhere, then a user whose history was [12, 12, 12] now has history [87, 87, 87]. Session autocorrelation still holds because Last seen predicts 87, and the true next value is still 87. The labels changed but the *structure* did not. This confirms that the baselines are robust to consistent label permutations.

6.2.3 Distribution-Shift Drift (Severe)

At row 100,000, for each subsequent event, with probability 0.8 the `parent_category` is overwritten with one of the 5 most popular pre-drift categories (selected uniformly at random). This simulates a drastic change in what categories are popular.

Observation: The Hybrid and baselines drop from ∼0.43 to ∼0.30 over 100,000 post-drift events. When a user's last category was 12 pre-drift but the actual post-drift value is now 561 (one of the top-5), Last seen predicts 12 and is wrong. The transition matrix's learned transitions also become stale. The ML models barely move because they were already at ∼0.13 — there is little further to fall.

Intent Drift Scenarios

6.3.1 Intent-Suppression Drift

At row 100,000, each subsequent event that is a `transaction` is flipped to a `view` with probability 0.5. This simulates a drop in conversion rate (e.g., a site outage affecting checkout).

Observation: The cart heuristic stays near ∼0.92 ROC-AUC because cart history is unaffected by the relabeling. ML models drift down slightly. F1 erodes for all models because there are fewer true positives to recall.

6.3.2 Intent-Boost Drift

At row 100,000, each subsequent event that is a `view` is flipped to a `transaction` with probability 0.05. This simulates a sudden surge in conversions (e.g., a promotional event).

Observation: All models drop, but the cart heuristic drops the most (from ∼0.92 to ∼0.60 ROC-AUC). The boost creates many fake transactions from users without cart history — exactly the

events the cart heuristic predicts as 0. ML models drop less (from ∼0.90 to ∼0.72) because they use richer features beyond just cart presence. This is the one scenario where ML clearly outperforms the heuristic baseline.

Drift Adaptation Mechanisms

ARF and HAT have built-in drift adaptation via ADWIN. ARF replaces entire trees in the ensemble when their error spikes; HAT replaces local subtrees at the node level. The stateful feature components (transition matrix, pattern miner, bipartite graph) adapt through fading windows rather than explicit detection. The Last seen baseline and the cart heuristic do not adapt to drift: Last seen simply copies the user's last value, and the cart heuristic is a static rule.

Conclusions

1. **Session autocorrelation dominates next-category prediction.** The Last seen baseline achieves ∼44% top-1 accuracy on a 283-class problem by exploiting the fact that 80% of consecutive events share the same category. Tree-based ML models cannot match this because axis-aligned splits cannot compactly express the rule "predict the value of a feature." This is a clean negative result for ML on this specific task and dataset.
2. **Fair evaluation requires scoring on all events.** Earlier versions of this project showed baselines at ∼86% accuracy because they silently abstained on cold-start events (51.3% of all events). The corrected evaluation (all models predict on all events, with trending fallback for cold start) reveals that baselines achieve ∼44%, not 86%.
3. **For intent prediction, ML adds real value on F1.** ARF Big with oversampling achieves F1 of 0.19 compared to 0.16 for the cart heuristic, while the heuristic leads on ROC-AUC (0.93 vs. 0.90). The ML models are more nuanced about when a purchase will occur.
4. **Drift response is task-specific.** Category-shuffle leaves all models unaffected; distribution-shift hurts baselines; intent-boost hurts the cart heuristic but ML adapts. No model tested is uniformly drift-resistant.